5 List Handling – Introduction

Lists First

Create a program called **lists_first** that creates a list of six integers. The list contains the following values: 1, 5, 4, 6, 2, and 7. The program should print the list on a single line as shown in the example output.

Note: Of course, you can use a loop in your program. But in Python, you can also print the whole list as required by calling the print function once. Please see in the course material how to format output when we are printing a list.

Example output:

```
1 5 4 6 2 7
```

Lists Second

Create a program called **lists_second**. The program should first input words from the user until the user decides to quit by entering "quit". Then the program should print the words in alphabetical order.

You can suppose that the user writes all words in lowercase.

Example output:

```
Enter a word (quit ends): white
Enter a word (quit ends): green
Enter a word (quit ends): blue
Enter a word (quit ends): gray
Enter a word (quit ends): quit
blue
gray
green
white
```

Lists Reverse

Create a program called **lists_reverse**. The program should first ask the user how many integers the user will enter. Then the program should input the integers from the user. Finally, the program should print the integers in ***reverse order*** on a single line as shown in the example output.

Example output:

```
How many integers? 4
Enter an integer: 20
Enter an integer: 18
Enter an integer: 22
Enter an integer: 14

14 22 18 20
```

Lists Integers

Create a program called **lists_integers** that first inputs five integers from the user and saves them to a list. Then the program should print the integers in ***descending order*** on a single line.

Example output:

```
Enter an integer: 1
Enter an integer: 5
Enter an integer: 2
Enter an integer: 4
Enter an integer: 3

5 4 3 2 1
```

Lists Info

Create a program called **lists_info** that inputs five integers from the user and saves them to a list. The program should print the following: the **count of the values** in the list, the **smallest value**, the **largest value,** and the **sum of the values** in the list.

Hint: Please notice that you can compute the required results by using Python's built-in functions.

Example output:

```
Enter an integer: 4
Enter an integer: 6
Enter an integer: 1
Enter an integer: 2
Enter an integer: 3

count: 5
min:   1
max:   6
sum:   16
```

Lists Grades

Create a program called **lists_grades** that first inputs a grade letter from user. Then the program should compute and show the percentage of the grade letter found in a list that contains the following values:

```
"A", "A", "B", "A", "C", "B", "C", "F", "B", "B", "A", "A", "C", "C", "C"
```

Example output:

```
Enter grade letter: B
26,7 %
```

Lists Surnames

Create a program called **lists_surnames**. The program should input surnames from the user until the user enters "ok". Then the program should print ***distinct surnames*** in alphabetical order as shown in the example output.

Please notice that we can use a *set* to remove duplicates.

Example output:

```
Enter a surname (ok ends): Jones
Enter a surname (ok ends): Harrison
Enter a surname (ok ends): Jones
Enter a surname (ok ends): Adams
Enter a surname (ok ends): Harrison
Enter a surname (ok ends): Harrison
Enter a surname (ok ends): ok

Adams, Harrison, Jones
```

Lists Positive Sum

Create a program called **lists_positive_sum**. The program should have a function named **positive_sum** that takes a list as argument. The function should return the sum of positive values in the list. If there are no positive values in the list, the function should return zero.

NB! The function should not modify the original list.

The *main function* should first input five integers from the user and save them to a list. Then the *main function* should call the **positive_sum** function and finally print the return value of the **positive_sum** function.

Example output:

```
Enter an integer: 1
Enter an integer: 2
Enter an integer: -3
Enter an integer: 4
Enter an integer: 5

12
```

Lists Words From URL

Create a program called **lists_words_from_url**. The program should first fetch a word list from the given URL and save the words to a list. This we can do with the code below[1].

```
from urllib.request import urlopen

url = "https://www.mit.edu/~ecprice/wordlist.10000"
word_list = urlopen(url).read().splitlines()
```

Then the program should determine and display the ***count of the words*** in the list, the ***average word length*** (float, no decimal-part formatting), and ***how many words there are of each length*** (the count of one-letter words, the count of two-letter words etc) as shown in the example output.

You can suppose that the *longest word contains 22 letters*. We can use the **len** function to get the length of a word. To create counters for words of different lengths, we can create another list and initialise it with 23 zeros as below.

```
counters = [0] * 23
```

The word lengths are between 1 and 22. You can use the word length as the index and leave the first element in the counters list untouched.

Your PC should be connected to the Internet when you are running this program on your own PC.

Example output:

```
10000 words
The average word length is 6.588
Length Count
     1    26
     2   396
     3   678
     4  1127
     5  1379
     6  1504
     7  1468
     8  1162
     9   909
    10   610
    11   379
    12   208
    13   101
    14    38
    15    10
    16     3
    17     0
    18     1
    19     0
    20     0
    21     0
    22     1
```

[1] (Additional information) On the website, the words are saved in a text file with 8-bit ASCII coding. Therefore, Python saves the words as byte arrays. If you want to print some of the words as normal strings, then you can convert the word to be printed to UTF-8 as follows: `word = word.decode("utf-8")`

NB! In the **rest of the list exercises**,

* You will be asked to write a specific function in each exercise.
* In Viope, there are ***unit tests***, which will ***test only your function*** instead of testing your whole program. Therefore, there is no example output given in the rest of list exercises. Instead of your program's output, Viope will show the unit test's output when you run your program in Viope.
* To prevent your whole program to be run when only one of its functions is being tested, please call your own main function in your program as below.

```
if __name__ == "__main__":
    main()
```

* In your main function, you can either create *a list with initial values*, or you can input values from the user and save them to your list before calling the other function.

Lists Sum Every Other

Create a program called **lists_sum_every_other**. The program should have a function named **sum_every_other** that takes a list as argument. The function should return the sum of every other value starting from the first value in the list. If the list is empty, the function should return **None**.

In Python, **None** is a keyword that represents null (no value at all)[2].

NB! The function should not modify the original list.

Lists Common Members

Create a program called **lists_common_members**. The program should have a function named **common_members** that takes two lists as arguments. The function should return **True** if the lists have at least one common member. Otherwise, the function should return **False**.

NB! The function should not modify the original lists.

Hint: In this exercise, you can solve the problem by writing a loop.

An alternative, advanced way to solve the problem is to use a set operation. Please see the Python documentation for more features of the set type: https://docs.python.org/3/library/index.html

Lists Count Distinct

Create a program called **lists_count_distinct**. The program should have a function named **count_distinct** that takes a list as argument. The function should return the count of distinct values in the list.

NB! The function should not modify the original list.

Hint: There is no need to write any loop in this exercise. See also the 'Lists Surnames' exercise where you have already printed *distinct* surnames.

[2] Under the hood, **None** refers to an object of the None type.

List Second Largest (*this is a grade level 4 exercise*)

Create a program called **lists_second_largest**. The program should have a function named **second_largest** that takes a list as argument. The function should return a copy of the second largest value in the list. If the list has no values, or if it has only one distinct value, the function should return **None**.

NB! The function should not modify the original list.

List Unique Integers

Create a program called **lists_unique_integers**. The program should first input five integers. Then the program should examine the inputted data and print the following:

- distinct integer values in ascending order
- all inputted values in ascending order.

Example output:

```
Enter an integer: 5
Enter an integer: 2
Enter an integer: 1
Enter an integer: 5
Enter an integer: 2
1, 2, 5
1, 2, 2, 5, 5
```

Lists Remove Successive Duplicates (*this is a grade level 4-5 exercise*)

Create a program called **lists_remove_successive_duplicates**. The program should have a function named **remove_successive_duplicates** that takes a list as argument. The function should remove successive duplicates from the original list.

We have a successive duplicate when two identical values exist one after each other in the list.

Original list: `[3, 1, 1, 2, 1, 2, 2, 2, 3]`
After the successive duplicates have been removed: `[3, 1, 2, 1, 2, 3]`

NB! This time, the function should modify the original list instead of returning a reference to a new list.